

CS246E—Assignment 5, Project (Fall 2025)

Henri K.

Due Dates: See project guidelines

DO NOT EVER SUBMIT TO MARMOSET WITHOUT COMPILING AND TESTING FIRST. If your final submission doesn't compile, or otherwise doesn't work, you will have nothing to show during your demo. Resist the temptation to make last-minute changes. They probably aren't worth it.

This project is intended to be doable in the allotted time. Because the breadth of students' abilities is quite wide, exactly what constitutes the amount of work that a student can do is difficult to nail down. Some students will finish quickly; others won't finish at all. We will attempt to grade this assignment in a way that addresses both ends of the spectrum. You should be able to pass the assignment with only a modest portion of your program working. Then, if you want a higher mark, it will take more than a proportionally higher effort to achieve, the higher you go. A perfect score will require a complete implementation. If you finish the entire program early, you can add extra features for a few extra marks.

Above all, **MAKE SURE YOUR SUBMITTED PROGRAM RUNS.** The markers do not have time to examine source code and give partial correctness marks for non-working programs. So, no matter what, even if your program doesn't work properly, make sure it at least does something.

Note: This assignment requires you to manipulate the terminal screen in a non-sequential way. Before you start programming, you will need to learn to use the Ncurses library (or equivalent functionality). You may also want to spend some time thinking about how you might make this library (which was written for C) more C++-friendly (e.g., build some abstractions around the core functionality).

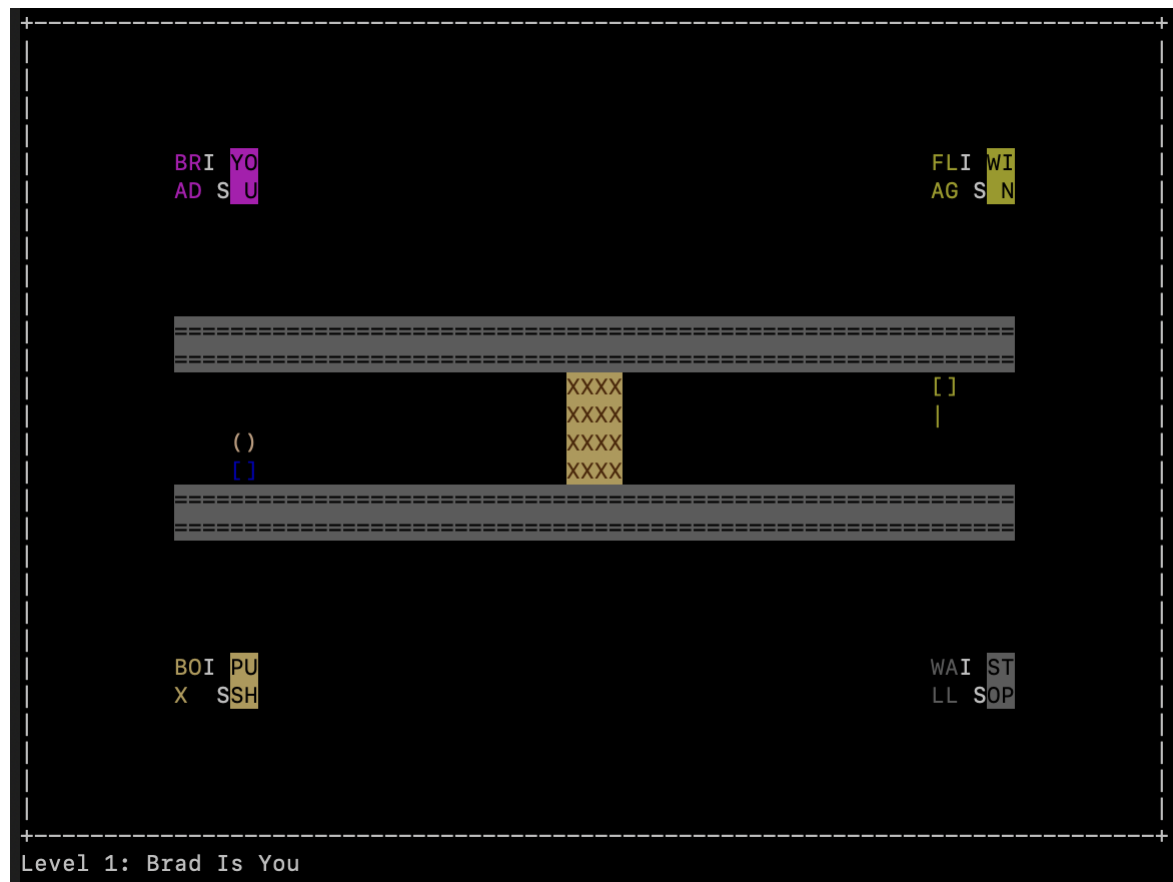
Brad is You

While the AGE project option tasks you with implementing a fully general game engine, Brad is You is a particular game that you can make for your project instead. While it is possible to create Brad is You using an implementation of AGE, it is likely easier to instead make a less general engine that is specially suited to this game. In either case this option is quite hard, and is recommended only for those seeking the challenge of implementing a highly complex game. Brad is You is a simplified clone of the 2019 puzzle game Baba is You by Hempuli. You can read about Baba is You on Wikipedia and/or watch a video of the game being played [here](#).

In Brad is You, you control a character and move around a grid. On the grid are various objects as well blocks of text that spell out sentences, and these sentences govern the rules of the game and the behaviour of each object. In particular, sentences can assign properties to objects, such YOU (meaning “is controlled by the player”) and PUSH (meaning “can be pushed”). By

pushing the blocks of text, the player can alter the logic of the game in real time, and must do so in order to each puzzle. The goal of each level is to win, which you do by making an object that is YOU touch an object that is WIN.

An Example Brad is You Screen



Basics

Game Display

Brad is You is played in a game area slightly larger than the AGE game area - 80 characters wide by 28 characters high, and surrounded by the same kind of border. However, the game is logically divided into a grid where each grid cell is 2x2, so the size of the grid is 40 x 14.

Beneath the border of the playing area, there is a single line of text, called the game text. By default, this text is white and is the name of the current level, but it is sometimes changed based on what happens in-game.

Gameplay and Controls

The gameplay of Brad is You is very simple: The player moves around on the grid one step at a time by pressing either the WASD keys or arrow keys, and can push some objects while other objects stop them from moving. The game's state updates only when the player presses a key. The player can press the spacebar to update the game state without moving. The game is split into

various levels, with the player starting at level 1. When the player wins, the game text changes to “Congratulations!” in yellow, and the game pauses for a moment before advancing to the next level.

The player can press R to restart the level, which sets the game text to change to “Restarting...” in white and pauses for a moment before resetting the level to its initial state. This is sometimes necessary, as it is possible for the player to ‘softlock’ themselves by making the game state such that it is impossible to win. The player can press backspace to exit the game and return to the terminal.

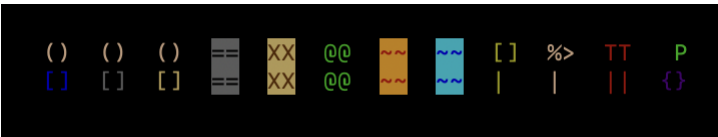
Game Objects and Logic

Entities

All entities in Baba is you are 2x2 squares of characters (some of which may be an invisible space character), and exist at a specific position on the grid. There are several different entities. The list of entities is:

BRAD ¹, SHAI ², GORD ³, WALL, BOX, BUSH, LAVA, H2O ⁴, FLAG, KEY, DOOR, PLUM.

These are their appearances in the same order as above:



Most entities have no animation, except for LAVA and H2O, which periodically alternate between two appearances to give the impression of a flowing liquid. You may add more entities, but you must have at least these ones. Your entities do not have to look exactly like these (you may change their characters, change or omit the colours, and/or give them different names), but every entity must be visually distinct from every other one.

Internally, each entity has a direction that it faces. When an object moves it always does so in the direction that it faces, so when the player inputs a move, their character first faces the direction specified by the key and then moves by one grid space in that direction. Note, however, that entities cannot move out of the playing area.

¹Yes, the same one teaching this course!
²A professor in the School of Computer Science.
³A professor emeritus in the School of Computer Science who taught my offering of CS 145 just before he retired.
⁴Due to the constraint of being in a terminal, we can't fit the 5 letters of “WATER” into a 2x2 square.

Properties

Brad is You is fundamentally different from most other videogames in that no entity is inherently different from any other entity except in appearance. How an entity behaves is entirely determined by what properties it has, and what properties it has is determined entirely by which sentences are active at a given time. If an entity has no properties then it is motionless and intangible, behaving no differently than if it were not there. The following is a description of every property:

- **YOU:** An entity that is YOU is controlled by the player, as described earlier. At the start of every level, at least one entity must be YOU, so that the player can do something. If at any point no entity is YOU, then the game must recognize this and have the game text switch to “Nothing is YOU. Restarting...” in red, before restarting the level as if the player had pressed R.
- **WIN:** The object of the game is to get an entity that is YOU to touch an entity that is WIN. (We say two entities are touching when they are on the same grid square.) When the player does so, they win and advance to the next level as described earlier.
- **STOP:** An entity that is STOP prevents other entities from moving through it. If an entity tries to move into an entity that is STOP, it remains in place and nothing happens. The edge of the playing area always behaves as if it is STOP.
- **PUSH:** An entity that is PUSH can be pushed by an entity that moves into its grid square (usually this is an entity that is YOU). The pushed object moves by one grid square in the direction that the pusher is facing, unless it collides with an entity that is STOP (in which case the pusher and the PUSH entity remain in place and nothing happens). Pushing can be chained in the following way: Suppose we have entities A, B, and C next to each other in a horizontal line, where A is YOU and B and C are PUSH. If the player moves right, A will push B and B will in turn push C, so all three entities move in the same direction. There is no limit to how many objects in a line can be ‘chain-pushed’ in this way. However, if the last entity collides with a STOP entity or the edge of the playing area, then none of them will move.
- **KILL:** An entity that is KILL is lethal to entities that are YOU: If an entity that is YOU touches an entity that is KILL, the YOU entity is destroyed. If this destruction makes it so that nothing is YOU, you must restart as described earlier.
- **MOVE:** An entity that is MOVE moves one step on its own every time the game updates. (Recall from the Basics section that the game’s state updates only when the player presses a key, and the player can press the spacebar to update the game state without moving.) If it collides with an entity that is STOP, it turns 180 degrees and begins moving in the opposite direction when the game next updates.
- **HOT and MELT:** These two properties work in relation to each other: If an entity that is HOT touches an entity that is MELT, the MELT entity is destroyed.
- **OPEN and SHUT:** These two properties work in relation to each other: If an entity that is OPEN touches an entity that is SHUT, both entities are destroyed.

You may add more properties, but you must have at least these ones.

Note that an entity is always considered to be touching itself. For example, if an entity has the properties HOT and MELT at the same time, it immediately melts itself and is destroyed.

Note also that it is possible for two different entities to occupy the same grid square at the same time without interacting. When this happens, one of them must fully obscure the other one because we cannot print two characters in the same location. Which one appears does not matter except for these two rules: An object that is YOU must always appear over an entity that is not YOU, and an object that has at least one property must appear over an entity that has no properties.

Words

In Brad is You, words are themselves a kind of entity that appear in each level as blocks of text, and they spell out sentences that determine which objects have which properties. Importantly, word entities are always implicitly PUSH, so YOU entities can always push them in order to rearrange said sentences.

There are five types of words: Nouns which represent entities, adjectives which represent properties, and the three special words IS, AND, and NOT. (Linguistically, IS and AND are ‘connectives’ and NOT is a ‘modifier’.) The following is the appearance of every word:



Words form a sentence when they are aligned next to each other on the grid in a straight (vertical or horizontal, not diagonal) line. Sentences read from left to right or top to bottom, but not right to left or bottom to top. A sentence may or may not be valid, i.e. have a meaning. When a valid sentence is formed, its meaning immediately takes effect in the game, e.g. when the sentence “SHAI IS YOU” is pushed into place, all SHAI entities in the level are immediately given the YOU property. Similarly, when a valid sentence is broken by having its words pushed out of place, its effect is immediately removed from the game.

Grammar

Sentences and Meanings

The grammar of Brad is You describes which sentences are valid, and what they mean. For instance, valid sentences include “BRAD IS YOU”, “BUSH IS KILL”, “BOX IS WALL”, and “NOT BUSH IS SHUT”, while invalid sentences include “AND IS YOU”, “OPEN IS SHUT”, “BRAD SHAI IS” and “NOT NOT NOT”.

Fundamentally, there is only kind of structure a valid sentence can have: “X IS Y”. But not every sentence of this form is valid. To describe which ones are, we introduce two new terms for

sequences of words: ‘subjects’ and ‘predicates’. Roughly speaking, a subject refers to one or more nouns, and a predicate refers to one or more adjectives.⁵ A subject is one of

1. N
2. S_1 AND S_2
3. NOT S

where N is a noun and S_1, S_2, S are subjects, and similarly a predicate is one of

1. A
2. P_1 AND P_2
3. NOT P

where A is an adjective and P_1, P_2, P are predicates.

In the case of a subject, N means all N entities in the level, S_1 AND S_2 means the union of all entities referred to by S_1 and S_2 , and NOT S means the set of all entities not referred to by S . (For example, “NOT LAVA” means “every entity in the level that is not LAVA”.) In the case of a predicate, A means the property A , P_1 AND P_2 means the union of all properties referred to by P_1 and P_2 , and NOT P refers to no properties but has a special negatory meaning described later.

Notice that from these definitions, NOTs can ‘stack’, and as in propositional logic they cancel each other when this happens. This means, for example, that “NOT NOT BRAD IS YOU” means the same thing as “BRAD IS YOU”, and “BOX IS NOT NOT NOT PUSH” means the same thing as “BOX IS NOT PUSH”.

With that, we return to the general sentence form “ X IS Y ”. For this sentence to be valid, X must be a subject and Y must either be a predicate or another subject. If Y is a predicate then the sentence means “The entities referred to by X have the properties referred to by Y ”. In the special case that Y is of the form NOT P , the sentence simply cancels any sentence of the form X IS P from taking effect. If Y is another subject then the sentence means “The entities referred to by X transform into the entities referred to by Y ”. You can see how various sentences behave by watching gameplay of Baba is You.

As a final note, there are many edge cases and complicated logical considerations that follow from these rules, and to implement them all perfectly would be an unreasonable ask. For the purposes of this project, you may assume a sentence never contains more than 7 words. If you have a question about an interaction of words that is not covered by this specification, the answer is that you can make your own reasonable choice, provided it is consistent with the specification and with your other choices.

⁵This isn’t exactly what subject and predicate mean in linguistics, but it’s close enough for our purposes.

Precedence

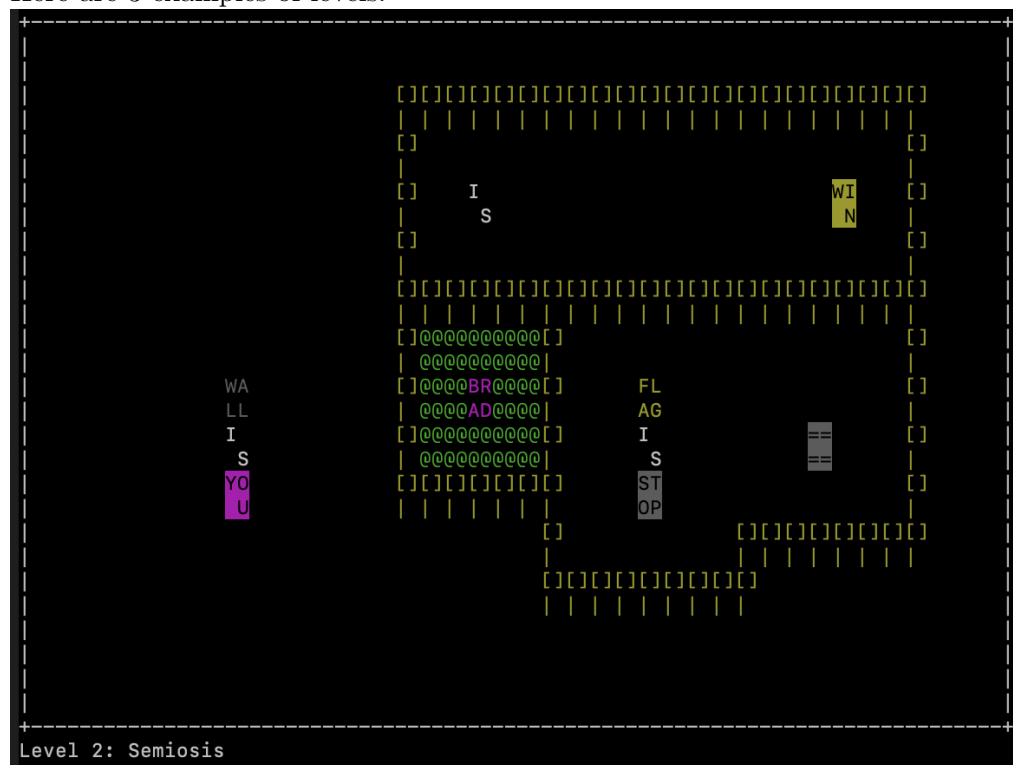
In some cases, it is possible for sentences with contradictory meaning to exist simultaneously. When this happens, one sentence takes overrides the other according to these precedence rules:

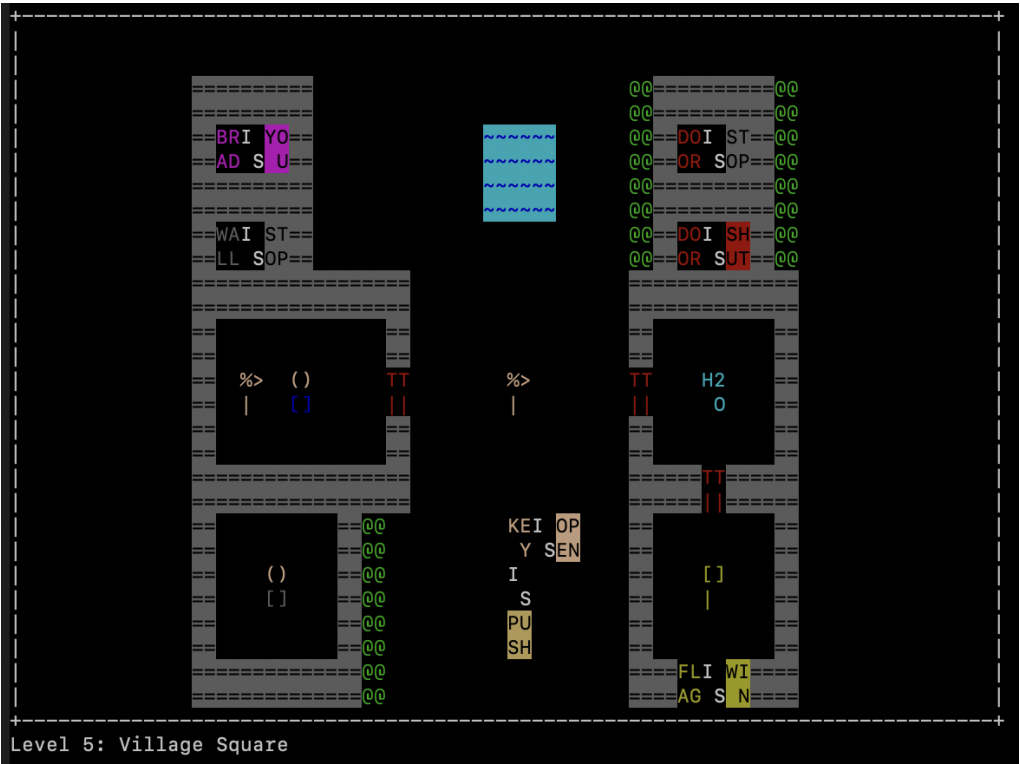
- For any distincts subjects X and Y , “ X is X ” overrides “ X is Y ”.
- For any subject X and predicate Y , “ X IS NOT Y ” overrides “ X IS Y ”.

Levels

Your implementation must have at least 10 levels, but you may include more if you wish. Each level must have a name to be displayed as the game text. You can copy level layouts from Baba is You or you can design your own, but they must all be winnable; i.e. you must be able to pass the level yourself during the demo. Every word in this specification must occur and be able to form a sentence in at least one level.

Here are 3 examples of levels:





Command Line Arguments

Brad is You takes one optional command line argument, which is a number from 1 to n where n is the number of levels in your implementation. The command line argument allows a player to skip to a certain level without playing the prior ones. For example, you can use the command line argument 3 to skip straight to the third level.

Architecture

Your implementation of this project should employ the MVC design architecture (loosely defined). Information on MVC will be given in a tutorial. As part of this requirement, you must separate the classes that manipulate the engine from those that dictate the game logic.

Grading

Your project will be graded as follows:

Correctness and Completeness	60%	Does it work? Does it implement all of the requirements?
Documentation	20%	Plan of attack; final design document.
Design	20%	UML; good use of separate compilation, good object-oriented practice; is it well-structured, or is it one giant function?

Even if your program doesn't work at all, you can still earn a lot of marks through good documentation and design, (in the latter case, there needs to be enough code present to make a reasonable assessment).

If Things Go Wrong

If you run into trouble and find yourself unable to complete the entire assignment, please do your best to submit something that works, even if it doesn't solve the entire assignment. For example:

- doesn't implement all words
- doesn't handle some types of pushing or movement
- doesn't handle all sentence types

You will get a higher mark for fully implementing some of the requirements than for a program that attempts all of the requirements, but doesn't run.

A well-documented, well-designed program that has all of the deficiencies listed above, but still runs, can still potentially earn a passing grade.

Plan for the Worst

Even the best programmers have bad days, and the simplest pointer error can take hours to track down. So be sure to have a plan of attack in place that maximizes your chances of always at least having a working program to submit. Prioritize your goals to maximize what you can demonstrate

at any given time. We suggest: Start with getting your program to display the game screen correctly. Then add support for entities and the YOU property. Then add the WIN property. Then continue to add the other properties, one-by-one. Then add the words and sentences. Parsing sentences and detecting when they are formed and broken may be the trickiest part, so may be best left until you have the properties themselves working. Take the time to work on a set of tests at the same time as you are writing your project. The main tests of your implementation will be the levels that you make, but you will need smaller tests throughout your development so that you can verify your work incrementally.

You will be asked to submit a plan, with projected completion dates, as part of your documentation for Due Date 1.

If Things Go Well

If you complete the entire project, you can earn up to 10% extra credit for implementing extra features. These should be outlined in your design document, and markers will judge the value of your extra features.

To earn significant credit, enhancements must be algorithmically difficult, or solve an interesting problem in object-oriented design. Trivial enhancements will not earn a significant fraction of the available marks.

Here are some suggestions for enhancements:

- Additional words from Baba is You (e.g. SINK, FLOAT, PULL, TELE, HAS, ON, TEXT, ALL, etc.)
- More Animations
- Move Undoing
- World Map
- Particle Effects
- Sound Effects

Submission Instructions

See **project_guidelines.pdf** for instructions about what should be included in your plan of attack and final design document.